

BA TASK 4 – WEEK 4

High-Performance Market Basket Analytics with Compressed FP-Trees from Scratch

1. Aim

To implement a frequent pattern mining system using a compressed FP-Tree from scratch and generate association rules from transactional data based on support and confidence.

2. Objective

- Understand frequent itemset mining.
- Build an FP-Tree using transaction data.
- Use node-link pointers to connect identical items.
- Construct conditional pattern bases and conditional FP-Trees.
- Find frequent itemsets recursively.
- Generate association rules using confidence.

3. Introduction

Market Basket Analysis is a data mining technique used to discover relationships between items purchased together. FP-Growth represents transactions using a compact Frequent Pattern Tree (FP-Tree). Unlike Apriori, it avoids repeatedly generating large candidate itemsets and instead mines the compressed tree recursively.

4. Important Concepts

Support: The proportion or count of transactions containing an itemset.

Confidence: The probability that a transaction containing the antecedent also contains the consequent.

FP-Tree: A compressed tree structure representing transaction data.

Node Link: A pointer connecting nodes containing the same item.

Conditional Pattern Base: Prefix paths associated with a selected item.

Conditional FP-Tree: A tree constructed from a conditional pattern base.

Association Rule: A relationship written as $X \rightarrow Y$.

5. Algorithm

1. Read the transaction dataset.
2. Count the frequency of every item.
3. Remove items below minimum support.
4. Sort remaining items by descending frequency.

5. Insert transactions into the FP-Tree.
6. Create node links between identical items.
7. Select items for mining.
8. Create conditional pattern bases.
9. Construct conditional FP-Trees.
10. Recursively mine frequent itemsets.
11. Generate association rules using confidence.

7. Sample Dataset

Transaction	Items
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Cola
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Cola

8. Program Explanation

FPNode class: Stores an item, count, parent, children and node-link pointer.

count_items(): Counts item frequency in the transaction database.

build_tree(): Filters infrequent items, orders transactions and creates the FP-Tree.

insert_transaction(): Adds ordered transactions and updates node links.

conditional_pattern_base(): Finds prefix paths leading to a selected item.

mine_tree(): Recursively mines conditional FP-Trees.

generate_rules(): Calculates confidence and creates association rules.

9. Expected Output

The program prints frequent itemsets such as Bread, Milk, Diaper and Beer, followed by frequent combinations such as Bread–Milk, Milk–Diaper and Diaper–Beer. It then prints rules whose confidence is at least 0.70, for example Bread → Milk and Beer → Diaper.

10. Performance and Compression

The FP-Tree compresses transactions by sharing common prefixes. Node-link pointers connect nodes containing the same item, making item traversal easier. Conditional FP-Trees reduce the search space during recursive mining. These features make FP-Growth efficient for frequent-pattern discovery.

11. Advantages

- Avoids repeated candidate generation.
- Compresses transaction data.
- Uses node links for efficient traversal.
- Mines patterns recursively.
- Useful for retail and recommendation analysis.

12. Applications

- Retail market basket analysis
- Product recommendation systems
- Cross-selling and up-selling
- Customer purchase behavior analysis
- E-commerce recommendation engines

13. Conclusion

The FP-Growth algorithm was implemented from scratch using a compressed FP-Tree. The program demonstrates frequency counting, FP-Tree construction, node-link management, conditional pattern mining and association-rule generation. It provides an efficient method for discovering useful purchasing patterns from transactional data.